Reinforcement learning problem and controls

Reinforcement learning is a problem where an agent tries to learn how to maximize its reward by interacting with its environment. Decisions, also known as controls, are important for reinforcement learning problems because they help agents learn which control in a given state is associated with the greatest rewards. There are many different reinforcement learning algorithms that help discover what are the best controls (those controls that lead to the greatest rewards); future tutorials, will cover these algorithms in greater depth.

Different reinforcement learning control algorithms can be combined to overcome some of their individual weaknesses. For example, reinforcement learning algorithms could encourage exploration early on during an agent's training in order to avoid getting stuck in local optima or suboptimal policies. In addition, reinforcement learning control algorithms can be used to help an agent recover from mistakes and return to the task at hand. Properly tuning reinforcement learning control selection is an important part of solving reinforcement learning problems. After an agent has been trained sufficiently, it would switch to exploiting its knowledge to obtain the greatest possible sum of rewards from any given state during the implementation or testing phase.

The set of all possible controls is referred to as the control space. This space, in most modeled problems is finite, especially for games. However, the control space can easily be infinite in problems where continuous controls are required. In reinforcement learning, the state-control space is often too large for the agent to search exhaustively, so it must use some form of approximation in order to focus its search. There are two main types of reinforcement learning algorithms: value-based and policy-based. Value-based methods aim to directly estimate the value function, which gives the expected return for taking a given control in a given state. Policy-based methods aim to directly learn a policy, which is a mapping from states to controls. Also, policies can be either deterministic or stochastic. Both types of reinforcement learning algorithms have their advantages and disadvantages, and there is significant ongoing research into developing new and improved reinforcement learning control algorithms.

Some Algorithms for control learning:

Some of the most common reinforcement learning control algorithms include:

- **Criterion of optimality:**
    - **Policy:** The agent's action selection is modeled as a map called policy:

$$\pi = A \times S \rightarrow [0,1]$$
$$\pi(a, s) = P_r(a_t = a | s_t = s)$$

      The policy map gives the probability of taking action a when in state s.There are also deterministic policies.

    - **State-value function:** The value function $V_\pi(s)$ is defined as, expected return starting with state s i.e.$s_0 = s$ and successively following policy π.Hence, roughly speaking, the value function estimates "how good" it is to be in a given state

- **Brute force:**
  The brute force approach entails two steps:

    - For each possible policy, sample returns while following it
    - Choose the policy with the largest expected return

  One problem with this is that the number of policies can be large, or even infinite. Another is that the variance of the returns may be large, which requires many samples to accurately estimate the return of each policy. These

problems can be ameliorated if we assume some structure and allow samples generated from one policy to influence the estimates made for others. The two main approaches for achieving this are value function estimation and direct policy search.

- **Value function:**

  Value function approaches attempt to find a policy that maximizes the return by maintaining a set of estimates of expected returns for some policy (usually either the "current" [on-policy] or the optimal [off-policy] one). These methods rely on the theory of Markov decision processes, where optimality is defined in a sense that is stronger than the above one: A policy is called optimal if it achieves the best-expected return from any initial state (i.e., initial distributions play no role in this definition). Again, an optimal policy can always be found amongst stationary policies. To define optimality in a formal manner, define the value of a policy π by

$$V^{\pi}(s) = E[R \mid s, \pi]$$

- **Direct policy search:**

  An alternative method is to search directly in (some subset of) the policy space, in which case the problem becomes a case of stochastic optimization. The two approaches available are gradient-based and gradient-free methods. Gradient-based methods (policy gradient methods) start with a mapping from a finite-dimensional (parameter) space to the space of policies: given the parameter vector θ, let π_{θ} denote the policy associated to θ. Defining the performance function by

$$\rho(\theta) = \rho^{\pi\theta}$$

  Policy search methods may converge slowly given noisy data. For example, this happens in episodic problems when the trajectories are long and the variance of the returns is large. Value-function based methods that rely on temporal differences might help in this case. In recent years, actor–critic methods have been proposed and performed well on various problems.

- **Monte Carlo methods:**
  Monte Carlo methods are reinforcement learning algorithms that can be used to solve both MDPs and Partially Observable MDPs (POMDPs). Monte Carlo methods are typically either on-policy or off-policy. On-policy reinforcement learning algorithms learn from experience sampled from the current policy, while off-policy reinforcement learning algorithms can learn from experience sampled from any arbitrary policy, even if that policy is not the optimal policy. Problems with this procedure include:

    1. The procedure may spend too much time evaluating a suboptimal policy.
    2. It uses samples inefficiently in that a long trajectory improves the estimate only of the single state-action pair that started the trajectory.
    3. When the returns along the trajectories have high variance, convergence is slow.
    4. It works in episodic problems only.
    5. It works in small, finite MDPs only.

- **Temporal difference methods:**
  Temporal difference (TD) methods are based on the recursive Bellman equation. The computation in TD methods can be
    - incremental (when after each transition the memory is changed and the transition is thrown away), or
    - batch (when the transitions are batched and the estimates are computed once based on the batch).

Batch methods, such as the least-squares temporal difference method, may use the information in the samples better, while incremental methods are the only choice when batch methods are infeasible due to their high computational or memory complexity. Some methods try to combine the two approaches. Temporal differences also overcome the issues of monte carlo method.

- **Function approximation methods:**
  In order to address the fifth issue of monte carlo method, function approximation methods are used. The algorithms adjust the weights, instead of adjusting the values associated with the individual state-action pairs. Methods based on ideas from nonparametric statistics (which can be seen to construct their own features) have been explored. Value iteration can also be used as a starting point, giving rise to the Q-learning algorithm and its many variants.

- **Model-based algorithms**

  Finally, all of the above methods can be combined with algorithms that first learn a model. For instance, the Dyna algorithm [Sutton, Richard (1990)] learns a model from experience, and uses that to provide more modelled transitions for a value function, in addition to the real transitions. Such methods can sometimes be extended to use of non-parametric models, such as when the transitions are simply stored and 'replayed' to the learning algorithm. There are other ways to use models than to update a value function.For instance, in model predictive control the model is used to update the behavior directly.